

My Week 2 AI Internship Journal

Building a RAG (Retrieval-Augmented Generation) system — explained simply, step by step

1. Quick Recap: Where Week 1 Left Off

In Week 1, I built a small web API using a tool called **FastAPI**. It had one main endpoint called **/ask** — basically a web address that, when you send it a question, it asks an AI model (like ChatGPT) and sends back an answer. It also tracked how many 'tokens' (pieces of words) were used and how much the request cost in dollars. I deployed this to a free hosting website called **Render**, so it had a real live URL anyone could use.

The problem: that **/ask** endpoint could only answer using what the AI already knew from its training. It had no way to answer questions about *my own* documents — like a company handbook or a policy file — because it never saw them. Week 2 is all about fixing that.

2. What I Actually Built This Week: RAG

RAG stands for **Retrieval-Augmented Generation**. In plain English: instead of just asking the AI a question and hoping it knows the answer, I first go and **fetch (retrieve)** the most relevant pieces of my own documents, hand those pieces to the AI as extra notes, and then ask it to answer *using only those notes*. It's like giving someone an open-book exam instead of asking them to answer from memory alone.

This matters because it lets the AI answer questions about things it was never trained on (like a private company policy), and it lets the AI honestly say 'I don't know' when the answer truly isn't in the documents, instead of making something up.

3. Step-by-Step: Everything I Did Today

Step 1 — Mapped my existing project with Claude Code

What I did: I asked Claude Code (an AI coding assistant) to read through my Week 1 project and explain how it worked, without changing anything yet.

Why I did it: Before adding new features, it's smart to understand what you already have. This avoided breaking Week 1's working code by accident.

Step 2 — Set up a Pinecone account and vector database

What I did: I created a free account on Pinecone, a service that stores 'embeddings' (explained below) so I can later search through my documents by meaning, not just keywords.

Why I did it: A normal database can't search by 'meaning' — only exact matches. Pinecone is built specifically to find the most similar pieces of text to a question.

Step 3 — Added the Pinecone connection code (infra layer)

What I did: Using Claude Code, I created a new file called `pinecone_store.py`. It had small helper functions: one to turn text into embeddings, one to connect to Pinecone, one to create the index if it doesn't exist, and one 'health check' to test the connection.

Why I did it: Building this in small, separate pieces (instead of one giant change) makes it much easier to test each part and fix bugs one at a time.

```
pip install -r requirements.txt

python pinecone_store.py
```

Step 4 — Fixed a tricky .env file bug

What I did: My app kept saying 'PINECONE_API_KEY is not set' even though I had added it to my `.env` file. Turned out the file wasn't actually saved (the editor tab still showed an unsaved dot). Once I properly saved it, the health check worked.

Why I did it: This taught me an important lesson: always double-check that a file is actually saved on disk, not just showing new content on screen.

```
python -c "print(open('.env', 'rb').read())"
```

Step 5 — Fixed a vector 'dimension' mismatch

What I did: My Pinecone index was set up with 512 dimensions, but my embedding model actually makes 1536-dimension vectors. Since the index had zero data in it yet, I simply deleted it and let my code recreate it with the correct size.

Why I did it: Think of dimensions like the number of measurements describing a shape. If my data has 1536 measurements but the storage box only expects 512, they simply won't fit together. Fixing it early (before adding real data) avoided a much messier fix later.

Step 6 — Built the /ingest endpoint

What I did: This is a new web address on my API. You send it some text and a `document_id`, and it: splits the text into small chunks, turns each chunk into an embedding, and stores it in Pinecone.

Why I did it: Before the AI can answer questions about a document, that document has to be broken into pieces and stored somewhere searchable. This endpoint is basically 'teaching' my app new information.

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/ingest" -Method Post -ContentType "application/json" -Body '{"document_id":"handbook","text": "..."}'
```

Step 7 — Tested retrieval BEFORE touching the AI (/debug/retrieve)

What I did: I built a separate, temporary endpoint that only searches Pinecone and returns the matching text chunks — it does NOT call the AI at all.

Why I did it: This is a golden rule in RAG projects: test that your search actually finds the right information before you let the AI use it. If I skip this and the AI gives a bad answer later, I won't know if the search was wrong or the AI was wrong.

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/debug/retrieve?q=How+many+days..."
```

Step 8 — Upgraded /ask to do full RAG

What I did: Now the real /ask endpoint: embeds the question, retrieves the top matching chunks from Pinecone, builds a special instruction (a 'grounding prompt') telling the AI to answer ONLY using those chunks and to cite the source, and finally calls the AI.

Why I did it: This is the heart of RAG. Instead of the AI guessing from memory, it's forced to read the retrieved notes first, and to say clearly where each fact came from.

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/ask" -Method Post -ContentType "application/json" -Body '{"question": "..."}'
```

Step 9 — Tested both a good answer and a refusal

What I did: I asked a question the documents could answer (got a correct, cited answer) and a question they couldn't (got an honest 'not enough information' refusal instead of a made-up answer).

Why I did it: A trustworthy AI system needs to know when to say 'I don't know.' Testing both cases proves the system isn't just guessing.

Step 10 — Ingested a bigger, realistic document

What I did: I fed in a full fictional company handbook (working hours, remote work policy, leave policy) and later an international student insurance policy, instead of just a one-line test sentence.

Why I did it: Testing with real, richer documents shows the system can handle actual use cases, not just a toy example.

Step 11 — Deployed everything to Render (made it live)

What I did: I pushed my updated code to GitHub, then added the new Pinecone environment variables to my Render web service, and Render automatically rebuilt and relaunched my app on its real internet address.

Why I did it: A project only really 'counts' as done when it works for anyone on the internet, not just on my own laptop. This step turns 'works on my machine' into 'works for real'.

```
git add .  
  
git commit -m "Add RAG: ingest, retrieval, and updated /ask"  
  
git push
```

Step 12 — Re-tested everything on the LIVE website

What I did: I ran the exact same ingest and ask tests again, but pointed at my real onrender.com web address instead of my laptop's local address.

Why I did it: Code can behave differently once deployed (missing settings, different environment). Testing live catches problems before anyone else finds them.

Step 13 — Built a Streamlit UI

What I did: I built a simple website (using a tool called Streamlit) with two tabs: one to paste in a document and ingest it, and one to type a question and see the answer, citations, and cost — all without needing to type terminal commands.

Why I did it: APIs are great for programs to talk to each other, but humans want to click buttons and see nice results, not type PowerShell commands. This UI makes the project usable by anyone.

```
python -m streamlit run streamlit_app.py
```

Step 14 — Wrote up my results and pulled in course materials

What I did: I created a document with screenshots explaining each part of the project, and I copied a folder of course reference materials (rag-vector-databases) from my instructor's GitHub repository into my own project using git clone and Copy-Item.

Why I did it: Documenting proves the work was actually done and helps me (or anyone else) remember how it works later. Pulling in course materials gives me extra examples to learn from going forward.

```
git clone https://github.com/akshika47/AI-Internship.git temp-course-repo
git pull
```

4. Tools I Used, and Why

Tool	Why I used it
Claude Code	An AI assistant that lives in my terminal and writes/edits code for me based on plain-English instructions. I used it to build every new file and endpoint this week instead of hand-typing all the code myself.
FastAPI	The Python framework my whole API is built with (also used in Week 1). It turns Python functions into web addresses (endpoints) other programs can call.
Pinecone	A cloud vector database — a specialized storage system built to hold embeddings and quickly find the most similar ones to a given question.
OpenAI Embeddings (text-embedding-3-small)	The model that converts text into embeddings (lists of numbers representing meaning). Used both when storing documents and when searching, so they can be compared fairly.
Render	A cloud hosting service (used since Week 1) that runs my API on the internet 24/7 so it's not just sitting on my own laptop.
Streamlit	A Python library for building simple, interactive websites very quickly, without needing to know HTML/CSS/JavaScript in depth.

Git & GitHub	Git tracks changes to my code over time; GitHub is a website that stores my code online and lets me share it or pull in other people's code (like the instructor's course materials).
PowerShell (terminal)	The command-line program on my Windows computer where I typed all the commands to run my server, test endpoints, and manage git.

5. New Words I Learned This Week (not used in Week 1)

Week 1 was mostly about calling an AI model directly. Week 2 introduced a whole new set of ideas around storing and searching documents. Here are the new terms, explained simply:

RAG (Retrieval-Augmented Generation)

A technique where the AI is given relevant document snippets before answering, instead of relying only on what it memorized during training.

Embedding

A list of numbers that represents the 'meaning' of a piece of text. Similar meanings produce similar number lists, which is what makes searching by meaning possible.

Vector database / Vector store

A special kind of database (like Pinecone) built to store embeddings and quickly find the ones most similar to a new piece of text.

Chunking

Splitting a long document into smaller pieces (chunks) before turning each piece into an embedding, since AI models work better with shorter pieces of text at a time.

Ingestion / to ingest

The process of taking a raw document, chunking it, embedding it, and storing it in the vector database so it can be searched later.

Dimension (of a vector)

The number of individual numbers in an embedding. Two embeddings must have the same dimension to be compared or stored together.

Similarity score

A number showing how closely two pieces of text match in meaning. Higher usually means more relevant.

Top-k retrieval

Asking the vector database for the 'k' most similar chunks to a question — for example, top-5 means 'give me the 5 best matches.'

Grounding prompt

A special instruction given to the AI that says 'only answer using the text below, and mention where each fact came from.' This keeps answers honest and traceable.

Citation / source attribution

Marking exactly which document a piece of an answer came from, so it can be double-checked.

Refusal

When the AI honestly says it doesn't have enough information to answer, instead of guessing or making something up.

Environment variable (.env file)

A private settings file that stores secret values (like API keys) outside of the actual code, so secrets are never accidentally shared or committed to GitHub.

API key

A secret password-like code that proves to a service (like Pinecone or OpenAI) that a request is really coming from me, and lets them track usage.

Health check

A small test built into the app that checks whether a connection (like to Pinecone) is working, without doing anything else.

Deploy / redeploy

Publishing code onto a live server (like Render) so it runs on the internet instead of just on a personal computer.

6. Where This Leaves Me

Week 1 gave me a live API that could answer general questions. Week 2 turned it into a real RAG system: it can now learn from my own documents, answer questions using only that information, clearly cite its sources, and honestly refuse when it doesn't know something — all deployed live on the internet with a simple point-and-click UI on top. This is the foundation most real-world AI assistants are actually built on.

Live project URL: <https://anirudh-ai-internship.onrender.com>